

BSE3211 – Assignment 1: Software Quality Attributes & System Testing

Due Date: March 27, 2026 11:59PM

- This is a group assignment (exercise) that requires that ALL members contribute to the outputs.
- Please ensure that submission (< 8 page report) is completed before the due date. You may want to submit a little earlier to avoid inconveniences of MUELE.
- This assignment exercises your understanding of most of the class concepts to this point. While we appreciate AI, please desist from its use in this assignment.
- Ensure that group participant details are captured correctly (Names, Student & Registration numbers)

Problem 1 – Software Quality Attributes [20 Points]

Refer to the Software Requirements Specification (SRS) document for the **Ride With Me** mobile application. The system allows car owners to share rides with other travelers for long journeys at a cost. You may add additional features or make decisions on how these features are implemented, as long as they fit the overall purpose of the system. In any case, state any assumptions that you make and describe any features you create.

For this system:

- a) Describe five (5) important software quality attributes.
- b) Create five (5) system-level tests.

Problem 2 - System Test Design [15 Points]

The tail utility is one of the most common command-line tools used in Mac OS, Ubuntu, and other Linux or Unix-based operating systems.

tail [ONE OR MORE OPTIONS] [FILENAMES OF ONE OR MORE FILES]

tail takes in one or more options and one or more filenames and then prints the last N (by default, 10) lines of the listed files to the screen.

Note: If you have a computer running Mac OS X, any Linux distribution, or the Windows Subsystem for Linux, you can open a command line and try this tool yourself. There are also browser-based Linux shells you can use to try the tool¹.

¹For example:

- <https://cocalc.com/doc/terminal.html>
- <https://bellard.org/jslinux/>
- <https://copy.sh/v86/?profile=linux26>
- https://www.tutorialspoint.com/unix_terminal_online.php

The documentation of the tool is included below. After reviewing the documentation and trying this tool, create system test cases using the process described in Lecture 4:

1. Given the explicit parameters (the list of options and the filename) and other environmental considerations (like the contents and existence of the file), identify the **choices** you control when testing. These are things that you can control that can affect the outcome of executing the function.

2. For each **choice**, Identify **representative input values**.

- a. These should not be concrete inputs (e.g., "4") but abstract types of input that could change the outcome (e.g., "a number > 0 and < size of the file")

Hint: A user can provide multiple options at once. As a starting point, you might want to look at the options as potential choices and the values that can be provided as representative values. Read the documentation carefully, and note that some options interact with other options or with the files provided. Constraints and representative values should reflect these interactions.

Tail Documentation tail [ONE OR MORE OPTIONS] [FILENAMES OF ONE OR MORE FILES]

Print the last 10 lines of each FILE to standard output. With more than one FILE, precede each with a header giving the file name. With no FILE, or when FILE is -, read standard input.

Note – when a flag uses capital letters for input (for example, --pid=PID), that means that the user supplies a value. When lower-case is used (for example, --follow=name), that means the literal word is used (name, in this case).

OPTIONS

- --bytes=K
 - output the last K bytes; or use --bytes +K to output bytes starting with the Kth of each file
- --follow[={name or descriptor}]
 - output appended data as the file grows; an absent option argument means 'descriptor'
- -F
- same as --follow=name --retry • --lines=K
 - output the last K lines, instead of the last 10; or use --lines +K to output starting with the Kth
- --max-unchanged-stats=N
 - with --follow=name, reopen a FILE which has not changed size after N (default 5) iterations to see if it has been unlinked or renamed (this is the usual case of rotated log files); with inotify, this option is rarely useful
- --pid=PID
 - with --follow, terminate after process ID, PID dies • --quiet
 - never output headers giving file names

- `--retry`
 - keep trying to open a file if it is inaccessible
- `--sleep-interval=N`
 - with `--follow`, sleep for approximately N seconds (default 1.0) between iterations; with `inotify` and `--pid=P`, check process P at least once every N seconds
- `--verbose`
 - always output headers giving file names
- `--help`
 - display this description text and exits
- `--version`
 - output version information and exit

If the first character of K (the number of bytes or lines) is a '+', print beginning with the Kth item from the start of each file, otherwise, print the last K items in the file. K may have a multiplier suffix: b 512, kB 1000, K 1024, MB 1000*1000, M 1024*1024, GB 1000*1000*1000, G 1024*1024*1024, and so on for T, P, E, Z, Y.

With `--follow`, tail defaults to following the file descriptor, which means that even if a tail'ed file is renamed, tail will continue to track its end. This default behavior is not desirable when you really want to track the actual name of the file, not the file descriptor (e.g., log rotation). Use `--follow=name` in that case. That causes tail to track the named file in a way that accommodates renaming, removal, and creation.

Problem 3 - System Test Design (with Postman) [15 Points]

Consider the following REST API: <http://dummy.restapiexample.com/> (there is more documentation at the URL). This API surfaces the following functions:

Route	Method	Example URL	Description
/employee	GET	http://dummy.restapiexample.com/api/v1/employees	Get all employee data
/employee/{id}	GET	http://dummy.restapiexample.com/api/v1/employee/1	Get data for a single employee ¹
/create	POST	http://dummy.restapiexample.com/api/v1/create	Create a new record in the database ²
/update/{id}	PUT	http://dummy.restapiexample.com/api/v1/update/21/	Update an employee record.
/delete/{id}	DELETE	http://dummy.restapiexample.com/api/v1/delete/2/	Delete an employee record.

1. If you do not have one already, create a free account at <https://www.postman.com/>
2. Use Postman to create tests for this API. Create 10 test cases in total, with at least one test case for each of these five API functions.
 - a. A test case has its own input (REST method + request body) and one or more assertions on the output (called “tests” in Postman).
 - b. Test cases should test both normal functionality as well as handling of erroneous input. Test cases may, optionally, also address performance aspects such as response time.
3. In your report, include the request type, body, assertions, and any other information that you used as part of your test cases.

You may use the process described in Lecture 4 (with choices, and representative values), but do not have to. Use your creativity and the test snippets provided by Postman to ensure that this API works as expected. Try error cases, like an ID of 0, and see if the API works as expected. You may want to consider adding tests to check for quality goals like performance as well.

¹ If you want to verify the normal results of /employee/{id}, use an ID between 1-24. IDs of 0 or higher than 24 are great for checking “error” cases.

² Note - the POST/PUT/DELETE methods do not actually make permanent changes in this example. You will get an appropriate response, but the record will not actually be created, updated, or deleted. You can use the result body to verify the “results” of running these three functions.